arXiv:submit/1110928 [stat.ML] 10 Nov 2014

Zero-bias autoencoders and the benefits of co-adapting features

Roland Memisevic
University of Montreal
Canada
roland.memisevic@umontreal.ca

Kishore Konda
Goethe University Frankfurt
Germany
konda@informatik.uni-frankfurt.de

David Krueger
University of Montreal
Canada
david.krueger@umontreal.ca

Abstract

Regularized training of an autoencoder typically results in hidden unit biases that take on large negative values. We show that negative biases are a natural result of using a hidden layer whose responsibility is to both represent the input data and act as a selection mechanism that ensures sparsity of the representation. We then show that negative biases impede the learning of data distributions whose intrinsic dimensionality is high. We also propose a new activation function that decouples the two roles of the hidden layer and that allows us to learn representations on data with very high intrinsic dimensionality, where standard autoencoders typically fail. Since the decoupled activation function acts like an implicit regularizer, the model can be trained by minimizing the reconstruction error of training data, without requiring any additional regularization.

1 Introduction

Autoencoders are popular models used for learning features and pretraining deep networks. In their simplest form, they are based on minimizing the squared error between an observation, $\boldsymbol{x}$, and a non-linear reconstruction defined as

$$\boldsymbol{r}(\boldsymbol{x}) = \sum_k h\big(\boldsymbol{w}_k^{\mathrm{T}}\boldsymbol{x} + b_k\big)\boldsymbol{w}_k + \boldsymbol{c} \tag{1}$$

where $\boldsymbol{w}_k$ and b_k are weight vector and bias for hidden unit k, $\boldsymbol{c}$ is a vector of visible biases, and $h(\cdot)$ is a hidden unit activation function. Popular choices of activation function are the sigmoid $h(a) = \big(1+\exp(-a)\big)^{-1}$, or the rectified linear (ReLU) $h(a) = \max(0, a)$. Various regularization schemes can be used to prevent trivial solutions when using a large number of hidden units. These include corrupting inputs during learning [26], adding a "contraction" penalty which forces derivatives of hidden unit activations to be small [23], or using sparsity penalties [3].

This work is motivated by the empirical observation that across a wide range of applications, hidden biases, b_k, tend to take on large negative values when training an autoencoder with one of the mentioned regularization schemes.

In this work, we show that negative hidden unit biases are at odds with some desirable properties of the representations learned by the autoencoder. We also show that negative biases are a simple consequence of the fact that hidden units in the autoencoder have the dual function of (1) selecting

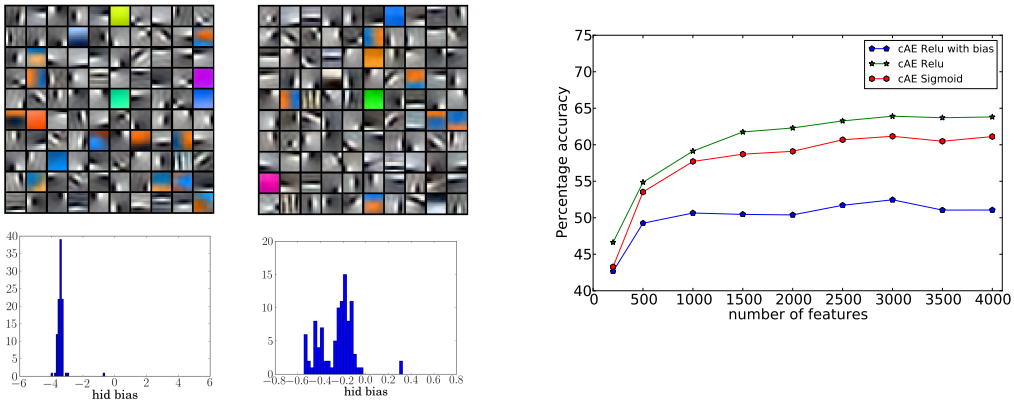


Figure 1: Left: Filters learned by a sigmoid contractive autoencoder [23] (contraction strength 1.0; left) and a ReLU denoising autoencoder [26] (zeromask-noise 0.5; right) from CIFAR-10 patches, and resulting histograms over learned hidden unit biases. Right: Classication accuracy on permutation invariant CIFAR-10 data using cAE with multiple different inference schemes. All plots in this paper are best viewed in color.

weight vectors which take part in reconstructing a given training point, and (2) representing the coefficients with which the selected reconstruct get combined to reconstruct the input (cf., Eq. 1).

To overcome the detrimental effects of negative biases, we then propose a new activation function that allows us to disentangle these roles. We show that this yields features that increasingly outperform regularized autoencoders in recognition tasks of increasingly high dimensionality. Since the regularization is "built" into the activation function, it allows us to train the autoencoder without additional regularization, like contraction or denoising, by simply minimizing reconstruction error. We also show that using an encoding without negative biases at test-time in both this model and a contractive autoencoder achieves state-of-the-art performance on the permutation-invariant CIFAR-10 dataset.[1]

1.1 Related work

Our analysis may help explain why in a network with linear hidden units, the optimal number of units tends to be relatively small [1, 14]. Training via thresholding, which we introduce in Section 3, is loosely related to dropout [6] in that it forces features to align with high-density regions. In contrast to dropout, our thresholding scheme is not stochastic. Hidden activations and reconstructions are a deterministic function of the input. Other related work is the work by [4] who introduce a variety of new activation functions for training autoencoders and argue for shrinking non-linearities, which set small activations to zero. In contrast to that work, we show that it is possible to train autoencoders without additional regularization, when using the right type of shrinkage function. Our work is also loosely related to [16] who discuss limitations of RBMs with *binary* observations.

2 Negative bias autoencoders

This work is motivated by the observation that regularized training of most common autoencoder models tends to yield hidden unit biases which are negative. Figure 1 shows an experimental demonstration of this effect using whitened 6×6-CIFAR-10 color patches [10]. Negative biases and sparse hidden units have also been shown to be important for obtaining good features with an RBM [13, 5].

[1] An example implementation of the zero-bias autoencoder in python is available at `http://www.iro.umontreal.ca/~memisevr/code/zae/`.

2.1 Negative biases are required for learning and bad in the encoding

Negative biases are arguably important for training autoencoders, especially overcomplete ones, because they help constrain capacity and localize features. But they can have several undesirable consequences on the encoding as we shall discuss.

Consider the effect of a negative bias on a hidden unit with "one-sided activation functions", such as ReLU or sigmoid (ie. activations which asymptote at zero for increasingly negative preactivation): On contrast normalized data it will act like a selection function, which zeroes out the activities for points whose inner product with the weight vector $\boldsymbol{w}_k$ is small. As a result, the region on the hypersphere that activates a hidden unit (ie. that yields a value that is significantly different from 0) will be a spherical cap, whose size is determined by the size of the weight vector and the bias. When activations are defined by spherical caps, the model effectively defines a radial basis function network on the hypersphere. (For data that is not normalized, it will still have the effect of limiting the number of training examples for which the activation function gets active.)

As long as the regions where weight vectors become active do not overlap this will be equivalent to clustering. In contrast to clustering, regions may of course overlap for the autoencoder. However, as we show in the following on the basis of an autoencoder with ReLU hidden units and negative biases, even where active regions merge, the model will resemble clustering, in that it will learn a point attractor to represent that region. In other words, the model will not be able to let multiple hidden units "collaborate" to define a multidimensional region of constant density.

2.1.1 Modes of the density learned by a ReLU autoencoder

We shall focus on autoencoders with ReLU activation function in the following. We add an approximate argument about sigmoid autoencoders in Section 2.1.2 below.

Consider points $\boldsymbol{x}$ with $r(\boldsymbol{x}) = \boldsymbol{x}$ which can be reconstructed perfectly by the autoencoder. The set of such points may be viewed as the mode of the true data generating density, or the true "manifold" the autoencoder has to find.

For an input $\boldsymbol{x}$, define the *active set* as those hidden units with positive response to $\boldsymbol{x}$:[2] $S(\boldsymbol{x}) = \{k : \boldsymbol{w}_k^{\mathrm{T}}\boldsymbol{x} + b_k > 0\}$. Let $W_{S(\boldsymbol{x})}$ denote the weight matrix restricted to the active units. In other words, $W_{S(\boldsymbol{x})}$ contains in its columns the weight vectors associated with active hidden units for data point $\boldsymbol{x}$. We can write the fixed point condition $r(\boldsymbol{x}) = \boldsymbol{x}$ for the ReLU autoencoder as

$$W_{S(\boldsymbol{x})}(W_{S(\boldsymbol{x})}^{\mathrm{T}}\boldsymbol{x} + \boldsymbol{b}) = \boldsymbol{x}, \tag{2}$$

or equivalently,

$$(W_{S(\boldsymbol{x})}W_{S\boldsymbol{x})}^{\mathrm{T}} - I)\boldsymbol{x} = -W_{S(\boldsymbol{x})}\boldsymbol{b} \tag{3}$$

This is a set of inhomogeneous linear equations, whose solutions are given by a specific solution plus the null-space of $M = (W_{S(\boldsymbol{x})}W_{S(\boldsymbol{x})}^{\mathrm{T}} - I)$. The null-space is given by the eigenvectors corresponding to the unity eigenvalues of $W_{S(\boldsymbol{x})}W_{S(\boldsymbol{x})}^{\mathrm{T}}$, whose number is equal to the number of orthonormal weight vectors in $W_{S(\boldsymbol{x})}$.

Learning in the absence of a bias, $\boldsymbol{b}$, will minimize the reconstruction error $\|\boldsymbol{x} - W_{S(\boldsymbol{x})}W_{S(\boldsymbol{x})}^{\mathrm{T}}\boldsymbol{x}\|^2$ will will enforce orthogonality of $W_{S(\boldsymbol{x})}$ for those hidden units that tend to be active together. Learning with a fixed, non-zero $\boldsymbol{b}$, on the other hand, will amount to minimizing reconstruction error between $\boldsymbol{x}$ and a shifted projection $\|\boldsymbol{x} - W_{S(\boldsymbol{x})}W_{S(\boldsymbol{x})}^{\mathrm{T}}\boldsymbol{x} + W_{S(\boldsymbol{x})}\boldsymbol{b}\|^2$, so that the orthonormal solution is no longer optimal: the projection has to account for the non-zero translation $W_{S(\boldsymbol{x})}\boldsymbol{b}$.

2.1.2 Sigmoid activations

The case of a sigmoid activation function is harder to analyze because the sigmoid is never exactly zero, and so the notion of an active set cannot be used. But we can characterize the manifold learned by a sigmoid autoencoder (and thereby an RBM, which learns the same density model [7])

[2] A similar approach of restricting attention to the active set was used recently by [22] to study the number of regions defined in a deep ReLU network

approximately using the binary activation $h_k(\boldsymbol{x}) = \big((\boldsymbol{w}_k^{\mathrm{T}}\boldsymbol{x}) + b_k \geq 0\big)$. The reconstruction function in this case would be

$$r(\boldsymbol{x}) = \sum_{k:\big((\boldsymbol{w}_k^{\mathrm{T}}\boldsymbol{x})+b_k \geq 0\big)} w_k$$

which is simply the superposition of active weight vectors (and hence not a multidimensional manifold either).

2.1.3 Zero-bias activations at test-time

This analysis suggests that the only role of non-zero biases in an autoencoder is to avoid trivial representations during *learning*, by setting them to large negative values as a result of regularization. While they are required, they do have a detrimental effect on the capacity of the model to learn a non-trivial density. As an implication, this suggests setting all biases to zero at test-time, where there is no reason to constrain the model capacity in any way.

In fact, [3, 24] recently showed that very good classification performance can be achieved using a linear classifier applied to a bag of features, using ReLU activation without bias. They also showed how this classification scheme is robust wrt. the choice of learning method used for obtaining features (in fact, it even works for random training points as features, or using K-means as the feature learning method).[3]

In Figure 1 we confirm this finding, and we show that it is still true when features represent whole CIFAR-10 images (rather than a bag of features). The figure shows the classification performance of a standard contractive autoencoder with sigmoid hidden units trained on the permutation-invariant CIFAR-10 training dataset (ie. using the whole images not patches for training), using a linear classifier applied to the hidden activations. It shows that much better classification performance (in fact the state-of-the-art in the permutation invariant task) is achieved when replacing the sigmoid activations used during training with a zero-bias ReLU activation at test-time (see Section 4 for more details).

3 Learning with threshold-gated activations

3.1 Thresholding linear responses

In light of the preceding analysis, hidden units should promote sparsity during learning, by becoming active in only a small region of the input space, but once a hidden unit is active it should use a *linear* not affine encoding. Furthermore, any sparsity-promoting process should be removed at test time.

To satisfy these criteria we suggest disentangling the *selection* function, which sparsifies hiddens, from the *encoding*, which defines the representation (and should be linear). To this end, the autoencoder reconstruction is defined as the product of the selection function and a linear representation:

$$\boldsymbol{r}(\boldsymbol{x}) = \sum_k h\big(\boldsymbol{w}_k^{\mathrm{T}}\boldsymbol{x}\big)\big(\boldsymbol{w}_k^{\mathrm{T}}\boldsymbol{x}\big)\boldsymbol{w}_k \tag{4}$$

The selection function, $h(\cdot)$, may use a negative bias to achieve sparsity, but once active a hidden unit uses a linear activation to define the coefficients in the reconstruction.

In our experiments we use the boolean selection function $h\big(\boldsymbol{w}_k^{\mathrm{T}}\boldsymbol{x}\big) = \big(\boldsymbol{w}_k^{\mathrm{T}}\boldsymbol{x} > \theta\big)$. This activation function is illustrated in Figure 2 (left). We will refer to it as Truncated Rectified (TRec) in the following. We set θ to 1.0 in most of our experiments (and all hiddens have the same threshold). While this is unlikely to be an optimal choice, we found it to work well and often on par with, or better than, traditional regularized autoencoders like the denoising or contractive autoencoder. Truncation, in contrast to the negative-bias ReLU, can also be viewed as a hard-thresholding operator, the inversion of which is fairly well-understood [2].

[3] In [3] the so-called "triangle activation" was used instead of a ReLU as the inference method for K-means. This amounts to setting activations below the mean activation to zero, and it is almost identical to a zero-bias ReLU since the mean linear preactivation is very close to zero on average.

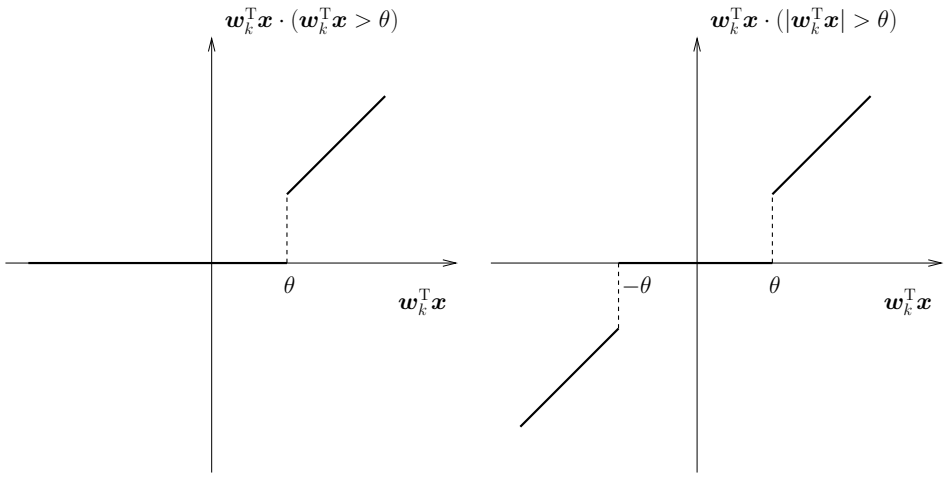


Figure 2: Activation functions for training autoencoders: thresholded rectified (left); thresholded linear (right).

Note that the TRec activation function is simply a peculiar activation function that we use for training. So training amounts to minimizing squared error without any kind of regularization. We drop the thresholding for testing, where we use simply the rectified linear response.

We also experiment with a "subspace" variant of the TRec activation function, defined as

$$\boldsymbol{r}(\boldsymbol{x}) = \sum_k h\big((\boldsymbol{w}_k^{\mathrm{T}}\boldsymbol{x})^2\big)\big(\boldsymbol{w}_k^{\mathrm{T}}\boldsymbol{x}\big)\boldsymbol{w}_k \tag{5}$$

It performs a linear reconstruction when the preactivation is either very large or very negative, so the active region is a subspace rather than a convex cone. We refer to this activation function as thresholded linear (TLin) below. See Figure 2 (right plot) for an illustration.

For both the TRec and TLin activation functions, the separation of the decision to fire from the encoding makes it possible to use a linear representation without bias. We shall refer to autoencoders with these activation as zero-bias autoencoder (ZAE) in the following.

3.2 Parseval autoencoders

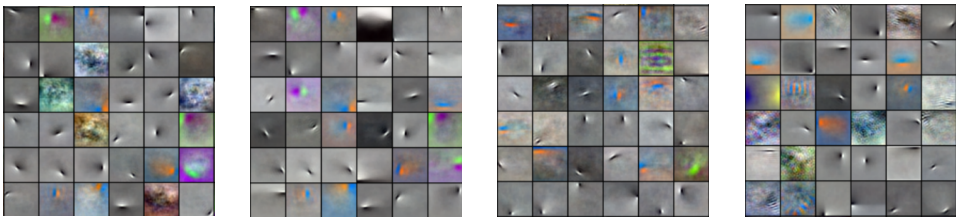
For overcomplete representations orthonormality can no longer hold. However, if the weight vectors span the data space, they form a *frame* (eg. [9]), so analysis weights $\tilde{\boldsymbol{w}}_i$ exist, such that an exact reconstruction can be written as

$$\boldsymbol{r}(\boldsymbol{x}) = \sum_{k \in S(\boldsymbol{x})} \big(\tilde{\boldsymbol{w}}_k^{\mathrm{T}}\boldsymbol{x}\big)\boldsymbol{w}_k \tag{6}$$

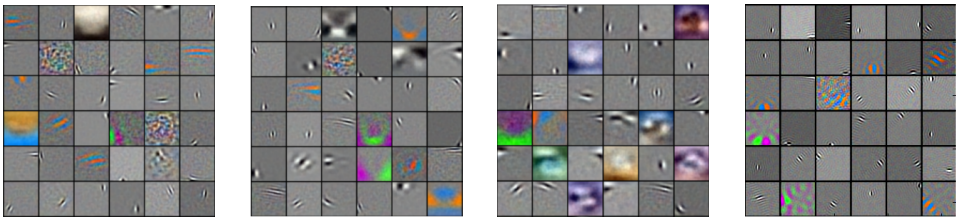
The vectors $\tilde{\boldsymbol{w}}_i$ and $\boldsymbol{w}_i$ are in general not identical, but they are related through a matrix multiplication: $\boldsymbol{w}_k = S\tilde{\boldsymbol{w}}_k$. The matrix S is known as frame operator for the frame $\{\boldsymbol{w}_k\}_k$ given by the weight vectors $\boldsymbol{w}_k$, and the set $\{\tilde{\boldsymbol{w}}_k\}_k$ is the dual frame associated with S [9]. The frame operator may be the identity in which case $\boldsymbol{w}_k = \tilde{\boldsymbol{w}}_k$ (which will be the case in an autoencoder with tied weights.)

Minimizing reconstruction error will make the frames $\{\boldsymbol{w}\}_k$ and $\{\tilde{\boldsymbol{w}}\}_k$ approximately duals of one another, so that Eq. 6 will approximately hold. More interestingly, for an autoencoder with tied weights, $\boldsymbol{w}_k = \tilde{\boldsymbol{w}}_k$, minimizing reconstruction error would let the frame approximate a Parseval frame [9], such that Parseval's identity holds:

$$\sum_{k \in S(\boldsymbol{x})} \big(\boldsymbol{w}_k^{\mathrm{T}}\boldsymbol{x}\big)^2 = \|\boldsymbol{x}\|^2 \tag{7}$$



(a) TLin AE (b) TRec AE (c) cAE RS=3 (d) cAE RS=-3



(e) TLin AE NW (f) TRec AE NW (g) cAE RS=3 NW (h) cAE RS=-3 NW

Figure 3: Features of different models trained on CIFAR-10 data. Top: PCA with whitening as preprocessing. Bottom: PCA with no whitening as preprocessing. RS denotes regularization strength.

4 Experiments

4.1 CIFAR-10

We chose the CIFAR-10 dataset [10] to study the ability of various models to learn from high dimensional input data. It contains color images of size 32×32 pixels that are assigned to 10 different classes. The number of samples for training is $50,000$ and for testing is $10,000$. We consider the permutation invariant recognition task where the method is unaware of the 2D spatial structure of the input. We evaluated several other models along with ours, namely contractive autoencoder, standout autoencoder [1] and K-means. The evaluation is based on classification performance.

The input data of size $3 \times 32 \times 32$ is contrast normalized and dimensionally reduced using PCA whitening retaining 99% variance. We also evaluated a second method of dimensionality reduction using PCA without whitening (denoted NW below). The number of features for each of model is set to $200, 500, 1000, 1500, 2000, 2500, 3000, 3500, 4000$. All models are trained with stochastic gradient descent. For all experiments in this section we chose a learning rate of 0.0001 for a few initial training epochs, and incremented it to 0.001 after some time. This is to ensure that scaling issues in the initializing are dealt with at the outset, and to help avoid any blow-ups during training. Each model is trained for 1000 epochs in total with a fixed momentum of 0.9. For inference, we use rectified linear units *without bias* for all the models. We classify the resulting representation using logistic regression with weight decay for classification, with weight cost parameter estimated using cross-validation on a subset of the training samples of size 10000.

The threshold parameter θ is fixed to 1.0 for both the TRec and TLin autoencoder. For the cAE we tried the regularization strengths $1.0, 2.0, 3.0, -3.0$; the latter being "uncontraction". In the case of the Standout AE we set $\alpha = 1$,$\beta = -1$. The results are reported in the plots of Figure 4. Learned filters are shown in Figure 3.

From the plots in Figure 4 it is observable that the results are in line with our discussions in the earlier sections. Note, in particular that the TRec and TLin autoencoders perform well even with very few hidden units. As the number of hidden units increases, the performance of the models which tend to "tile" the input space tends to improve.

In a second experiment we evaluate the impact of different input sizes on a fixed number of features. For this experiment the training data is given by image patches of size P cropped from the center of each training image from the CIFAR-10 dataset. This yields for each patch size P a training set of 50000 samples and a test set of 10000 samples. The different patch sizes that we evaluated are

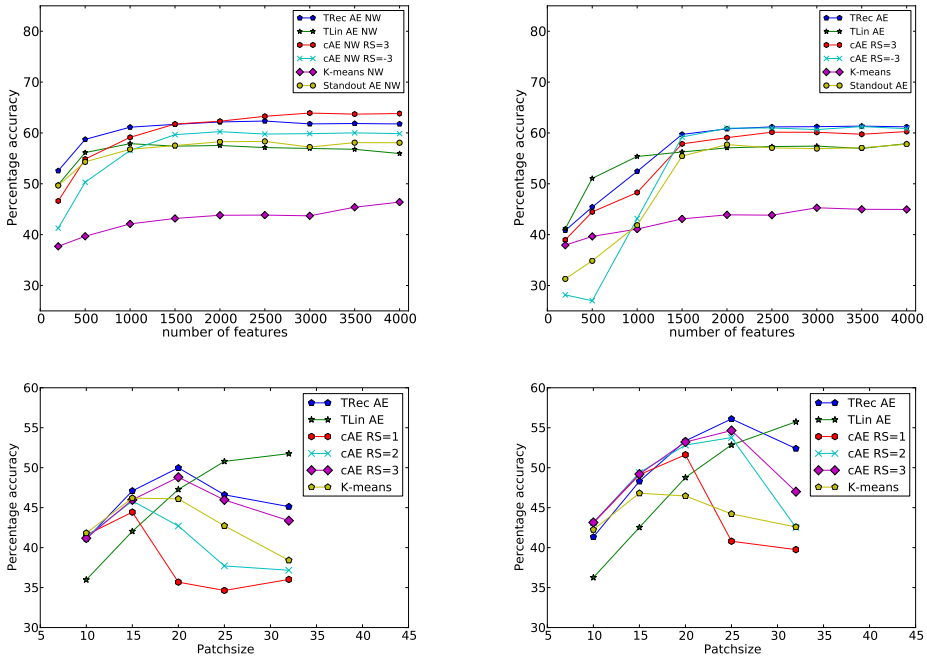


Figure 4: Top row: Classification accuracy on permutation invariant CIFAR-10 data as a function of number of features. PCA with whitening (left) and without whitening (right) is used for preproceesing. Bottom row: Classification accuracy on CIFAR-10 data for 500 features (left) 1000 features (right) as a function of input patch size. PCA with whitening is used for preprocessing.

$10, 15, 20, 25$ as well as the original image size of 32. The number of features is set to 500 and 1000. The same preprocessing (whitening/no whitening) and classification procedure as above are used to report performance. The results are shown in Figure 4.

When using preprocessed input data directly for classification, the performance increased with increasing patch size P, as one would expect. Figure 4 shows that for smaller patch sizes, all the models perform equally well. The performance of the TLin based model improves monotonically as the patch size is increased. All other model's performances suffer when the patch size gets too large. Among these, the ZAE model using TRec activation suffers the least, as expected.

We also experimented by initializing a neural network with features from the trained models. We use a single hidden layer MLP with ReLU units where the input to hidden weights are initialized with features from the trained models and the hidden to output weights from the logistic regression models (following [10]). A hyperparameter search yielding 0.7 as the optimal threshold, along with supervised fine tuning helps increase the best performance in the case of the TRec AE to 63.8. The same was not observed in the case of the cAE where the performance went slightly down. Thus using the TRec AE followed by supervised fine-tuning yields 63.8% accuracy and the cAE with regularization strength of 3.0 yields 63.9%. To the best of our knowledge both results beat the current state-of-the-art performance on the permutation invariant CIFAR-10 recognition task (cf., for example, [11]), with the cAE with a large number of hiddens slightly outperforming the TRec. In both cases PCA without whitening was used as preprocessing. In contrast to [10] we do not train on any extra data, so none of these models is provided with any knowledge of the task beyond the preprocessed training set.

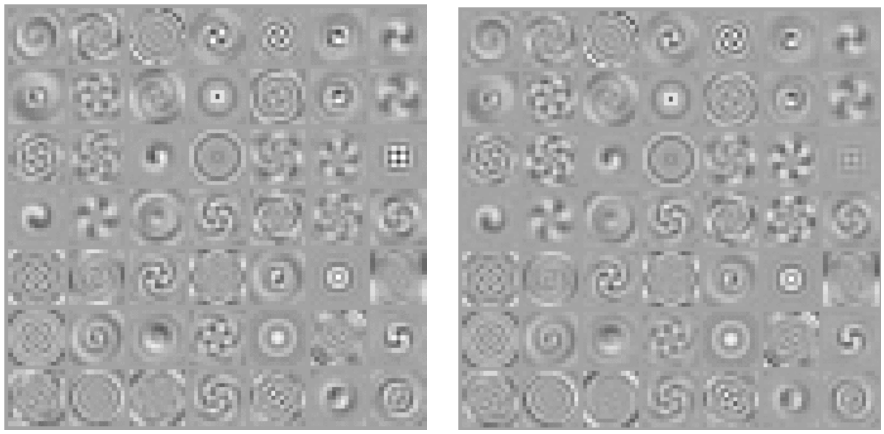
4.2 Video data

An dataset with very high intrinsic dimensionality are videos that show transforming random dots, as used in [19] and subsequent work: each data example is a vectorized video, whose first frame is a random image and whose subsequent frames show transformations of the first frame. This type of

data has an intrinsic dimensionality which is at least as high as the dimensionality of the first frame. So it is very high if the first frame is a random image.

It has been widely assumed that only bi-linear models, such as [19] and related models, would be able to learn useful representations of this data. The interpretation of this data in terms of high intrinsic dimensionality suggests that a simple autoencoder should be able to learn reasonable features, as long as it uses a linear activation function so hidden units can span larger regions.

We found that this is indeed the case by training the ZAE on rotating random dots as proposed in [19]. Figure 5 depicts filters learned from 10-frame random dot videos and shows that the model learns to represent the structure in this data by developing phase-shifted rotational Fourier components as discussed in the context of bi-linear models. We were not able to learn features that were distinguishable from noise with the cAE, which is in line with existing results (eg. [19]).



Model	Average precision
TRec AE	50.4
TLin AE	49.8
covAE [17]	43.3
GRBM [25]	46.6
K-means	41.0
contractive AE	45.2

Figure 5: Top: Subset of filters learned from rotating random dot movies (frame 2 on the left, frame 4 on the right). Bottom: Average precision on Hollywood2.

We then chose activity recognition to perform a quantitative evaluation of this observation. The intrinsic dimensionality of real world movies is probably lower than that of random dot movies, but higher than that of still images. We used the recognition pipeline proposed in [12, 8] and evaluated it on the Hollywood2 dataset [15]. The dataset consists of 823 training videos and 884 test videos with 12 classes of human actions. The models were trained on PCA-whitened input patches of size $10 \times 16 \times 16$ cropped randomly from training videos. The number of training patches is $500,000$. The number of features is set to 600 for all models.

In the recognition pipeline, sub blocks of the same size as the patch size are cropped from $14 \times 20 \times 20$ super-blocks, using a stride of 4. Each super block results in 8 sub blocks. The concatenation of sub block filter responses is dimensionally reduced by performing PCA to get a super block descriptor, on which a second layer of K-means learns a vocabulary of spatio-temporal words, that get classified with an SVM (for details, see [12, 8]).

In our experiments we plug the features learned with the different models into this pipeline. The performances of the models are reported in Figure 5 (right). They show that the TRec and TLin autoencoders clearly outperform the more localized models. Surprisingly, they also outperform more sophisticated gating models, such as [19]. This may suggest viewing gating itself as a way to obtain a linear encoding as we discuss in the next section.

4.3 Rectified linear inference

In previous sections we discussed the importance of (unbiased) rectified linear inference. Here we experimentally show that using rectified linear inference yields the best performance among different inference schemes. We use a cAE model with a fixed number of hiddens trained on CIFAR-10 images, and evaluate the performance of

1. Rectified linear inference with bias (the natural preactivation for the unit): $[W^T X + b]_+$
2. Rectified linear inference without bias: $[W^T X]_+$
3. natural inference: $\mathrm{sigmoid}(W^T X + b)$

The performances are shown in Figure 1 (right), confirming and extending the results presented in [3, 24].

5 Discussion

Quantizing the input space with tiles proportional in quantity to the data density is arguably the best way to represent data given enough training data and enough tiles, because it allows us to approximate any function reasonably well using only a subsequent linear layer. However, for data with high intrinsic dimensionality and a limited number of hidden units, we have no other choice than to summarize regions using responses that are invariant to some changes in the input. Invariance, from this perspective, is a necessary evil and not a goal in itself. But it is increasingly important for increasingly high dimensional inputs.

We showed that linear not affine hidden responses allow us to get invariance, because the density defined by a linear autoencoder is a superposition of (possibly very large) regions or subspaces.

After a selection is made as to which hidden units are active for a given data example, linear coefficients are used in the reconstruction. This is very similar to the way in which gating and square pooling models (eg., [20, 18, 19, 21, 12]) define their reconstruction: The response of a hidden unit in these models is defined by multiplying the filter response or squaring it, followed by a nonlinearity. To reconstruct the input, the output of the hidden unit is then multiplied by the filter response itself, making the model bi-linear. As a result, reconstructions are defined as the sum of feature vectors, weighted by *linear* coefficients of the active hiddens. This may suggest interpreting the fact that these models work well on videos and other high-dimensional data as a result of using linear, zero-bias hidden units, too.

References

[1] Jimmy Ba and Brendan Frey. Adaptive dropout for training deep neural networks. In *Advances in Neural Information Processing Systems*, pages 3084–3092, 2013.

[2] Holger Boche and Friedrich Philipp Mijail Guillemard and, Gitta Kutyniok and. Signal recovery from thresholded frame measurements. In *SPIE 8858, Wavelets and Sparsity XV*, August 2013.

[3] Adam Coates, Honglak Lee, and A. Y. Ng. An analysis of single-layer networks in unsupervised feature learning. In *Artificial Intelligence and Statistics*, 2011.

[4] Rotislav Goroshin and Yann LeCun. Saturating auto-encoders. In *International Conference on Learning Representations (ICLR2013)*, April 2013.

[5] Geoffrey Hinton. A Practical Guide to Training Restricted Boltzmann Machines. Technical report, University of Toronto, 2010.

[6] Geoffrey E. Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *CoRR*, abs/1207.0580, 2012.

[7] Hanna Kamyshanska and Roland Memisevic. On autoencoder scoring. In *Proceedings of the 30th International Conference on Machine Learning (ICML 2013)*, 2013.

[8] Kishore Reddy Konda, Roland Memisevic, and Vincent Michalski. The role of spatio-temporal synchrony in the encoding of motion. In *International Conference on Learning Representations (ICLR2014)*, 2014.

[9] J. Kovacevic and A. Chebira. *An Introduction to Frames*. Foundations and trends in signal processing. Now Publishers, 2008.

[10] Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. *Master's thesis, Department of Computer Science, University of Toronto*, 2009.

[11] Quoc Le, Tamas Sarlos, and Alex Smola. Fastfood - approximating kernel expansions in loglinear time. In *30th International Conference on Machine Learning (ICML)*, 2013.

[12] Q.V. Le, W.Y. Zou, S.Y. Yeung, and A.Y. Ng. Learning hierarchical invariant spatio-temporal features for action recognition with independent subspace analysis. In *CVPR*, 2011.

[13] Honglak Lee, Chaitanya Ekanadham, and Andrew Ng. Sparse deep belief net model for visual area v2. In *Advances in Neural Information Processing Systems 20*, 2008.

[14] Alireza Makhzani and Brendan Frey. k-sparse autoencoders. *CoRR*, abs/1312.5663, 2013.

[15] Marcin Marszałek, Ivan Laptev, and Cordelia Schmid. Actions in context. In *IEEE Conference on Computer Vision & Pattern Recognition*, 2009.

[16] James Martens, Arkadev Chattopadhyay, Toniann Pitassi, and Richard Zemel. On the representational efficiency of restricted boltzmann machines. In *Neural Information Processing Systems (NIPS) 2013*, 2013.

[17] Roland Memisevic. Gradient-based learning of higher-order image features. In *ICCV*, 2011.

[18] Roland Memisevic and Geoffrey Hinton. Unsupervised learning of image transformations. In *CVPR*, 2007.

[19] Roland Memisevic and Geoffrey E. Hinton. Learning to represent spatial transformations with factored higher-order boltzmann machines. *Neural Computation*, 22(6):1473–1492, June 2010.

[20] Bruno Olshausen, Charles Cadieu, Jack Culpepper, and David Warland. Bilinear models of natural images. In *SPIE Proceedings: Human Vision Electronic Imaging XII*, San Jose, 2007.

[21] Marc'Aurelio Ranzato, Alex Krizhevsky, and Geoffrey E. Hinton. Factored 3-Way Restricted Boltzmann Machines For Modeling Natural Images. In *Artificial Intelligence and Statistics*, 2010.

[22] Yoshua Bengio Razvan Pascanu, Guido Montufar. On the number of inference regions of deep feed forward networks with piece-wise linear activations. *CoRR*, arXiv:1312.6098, 2014.

[23] Salah Rifai, Pascal Vincent, Xavier Muller, Xavier Glorot, and Yoshua Bengio. Contractive Auto-Encoders: Explicit Invariance During Feature Extraction. In *ICML*, 2011.

[24] Andrew Saxe, Pang Wei Koh, Zhenghao Chen, Maneesh Bhand, Bipin Suresh, and Andrew Ng. On random weights and unsupervised feature learning. In *Proceedings of the 28th International Conference on Machine Learning*, 2011.

[25] Graham W. Taylor, Rob Fergus, Yann LeCun, and Christoph Bregler. Convolutional learning of spatio-temporal features. In *Proceedings of the 11th European conference on Computer vision: Part VI*, ECCV'10, 2010.

[26] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning*, 2008.